MAE 6291
Internet of Things for Engineers

Prof. Kartik Bulusu, MAE Dept.

Week 6 [02/25/2026]	• Recap: Layers in IoT systems - 3 level layer model • Sensors • Sensor types and considerations • The 5-layer IoT system model • Automating an email feature – Minimalist application layer in your IoT product design	• Discussion on Midterm project expectations • In-class email automation using yagmail and smtplib • In-class Flask API development • Raspberry Pi messaging with Python scripts [Graded Lab Activity] • Proximity sensing on Boot [Demo] • Recap The Linux Terminal

git clone https://github.com/gwu-mae6291-iot/spring2026_codes.git

GW School of Engineering & Applied Science
THE GEORGE WASHINGTON UNIVERSITY

Spring 2026

Photo: Kartik Bulusu

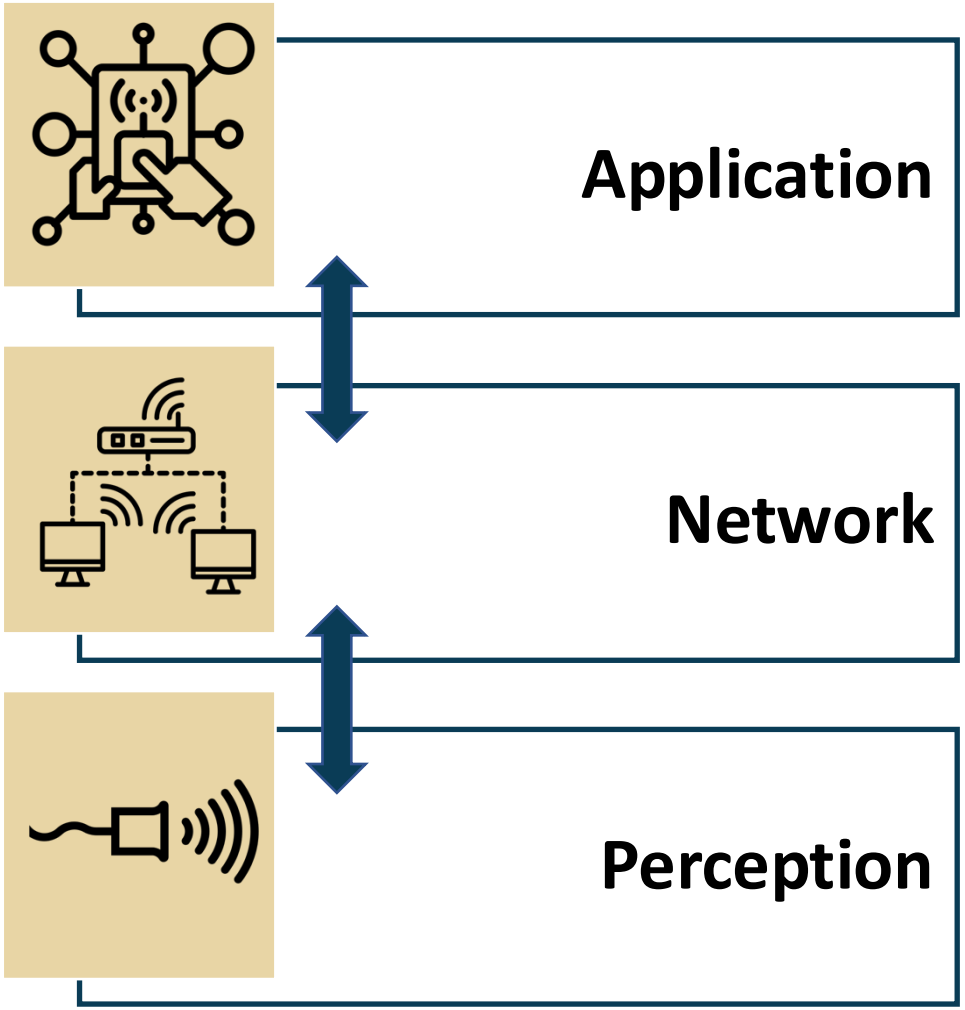
Content Attribution & Sources

This course material incorporates both original content and supplementary visual aids:

- **Images with references:** Properly cited in slide captions, figure legends, or in-text attributions
- **Unreferenced images:** Sourced from Perplexity.io and used for illustrative purposes in an educational context
- **Fair use:** All materials are used in compliance with educational fair use guidelines

The 3-Layer IoT Architecture

Icon Sources:
sensor by Carolina Cani:, sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo: https://thenounproject.com/browse/icons/term/sensor
wifi network by Matthias Hartmann:: https://thenounproject.com/browse/icons/term/wifi-network/
application by Chaowalit Koetchuea: https://thenounproject.com/browse/icons/term/application/



Network layer

Guest lecture

Protocol for point-to-point communication between two devices

by
Jitish Kolanjery
Sr. Software Engineer
Google Inc.

Congratulations on running you first IoT program last week!

Midterm projects
- open discussions on how to proceed

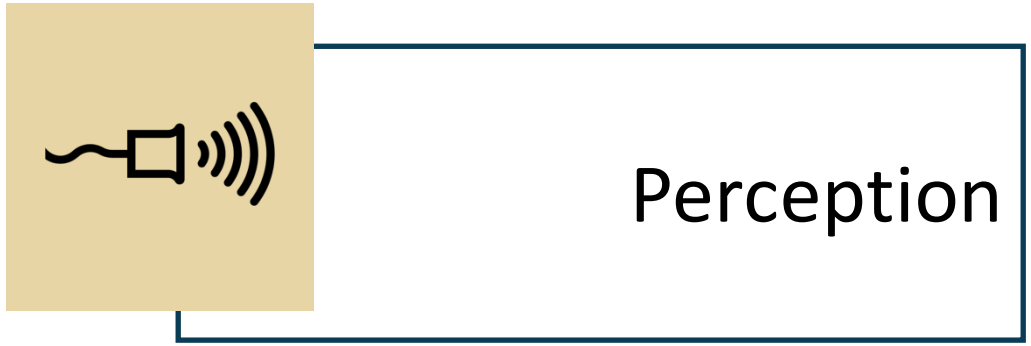
Midterm Project Status – Spring 2026

Name	Project title	Hardware requirements	Status
Ali Ahmed Mourad-Aziz	***Neocoast***	MPU6050 Accelerometer and Gyroscope Module, Photoresistor Module, RGB LED Module, Plastic Enclosure, Velcro Mounting Strips	**Approved. Needs to collect sensors**
Angie Abd Al Rahman	***Glowabetes***	RGB LED Module	**Approved. Needs to collect sensors**
Julia McGillivray, Claudia S. Rodriguez Chevere	***Plant Sense***	Photoresistors, Raindrop sensor, Temperature sensor, Moisture sensor	**Approved. Needs to collect sensors**
Farah Salaheddine	***Real-Time Smart Study Room Occupancy and Noise Monitoring System Using Raspberry Pi***	HC-SR04 Ultrasonic Sensor, Sound Sensor, Humiture Sensor, I2C LCD 1602 Module, RGB LED, Relay Module	**Approved. Needs to collect sensors**
Kendall Cosper	***PRAK (Pattern Reading Assistant for Knitters)***	I2C LCD 1602 Module	**Approved. Needs to collect sensors**
Jacob Ethan Dela Rosa-Frio	***EscapeBro***	Magnetic reed switches, Push buttons Light sensors, Tilt sensors or accelerometers RFID module	**Approved. Needs to collect sensors**
Jairo Sanchez	***Ball Mill Timing System***	Sparkfun Reed Switch I2C LCD 1602 Module Rotary Potentiometer	**Approved. Needs to collect sensors**
James Clark Ashby III	***What2Wear***	Temperature sensors, camera	**Approved. Needs to collect sensors**
Mathew Chapin	***Smart Alarm***	Buzzer, ultrasonic proximity sensor	**Approved. Needs to collect sensors**
Julian O'Neill	***Flood Warning System-With push notification***	ESP32, Capacitive Analog Soil Moisture Sensor, TP4056 Type C USB Charger Module, 3.7V Lipo Batter 1000mAh	**Approved. Needs to collect actuators**
Maya Lerma	***Smart Indoor Air Quality Monitor***	Temperature sensor , Humidity sensor , Gas sensor LED, Buzzer	**Approved. Needs to collect sensors**
Tala Hawa	***Visual LED Aid of Joysticks Speed for Robotic Arms Utilized in Surgeries***	RGB LED, Joystick PS2	**Approved. Needs to collect sensors**
Tom Li	***Strike Tracker***	MPU6050 IMU, ESP32 Microcontroller, Fitgriff Heavy-Duty Wrist Straps, 3.7V LiPo Battery or Small USB Battery Pack	**Approved. Needs to collect sensors**
Treesta James	***HerCycle LED: A Raspberry Pi-Based Menstrual Cycle Signaling System***	Microphone	**Approved. Needs to collect sensors**
Valeria C. Tovar-Munoz	***Smart Reading Analytics for Physical Books***	IR Obstacle Sensor, Hall Switch, or MPU6050 Module. It could be a combination of an IR Obstacle sensor + MPU6050 Module	**Approved. Needs to collect sensors**
Kartik Bulusu , Eliot Hunter	***Didn't provide a title***	**They are not sure!**	**Unclear how they are going to pull this off!!**

Graded deliverables:

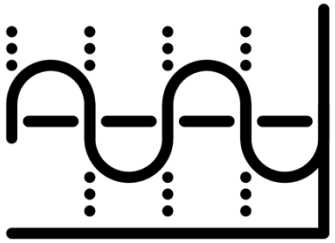
1. Midterm project demo
2. Midterm project presentation
3. 2-minute video of your working project
4. Github repo of your codes
5. Midterm project report in a conference-style template

Sensors and the IoT perception layer



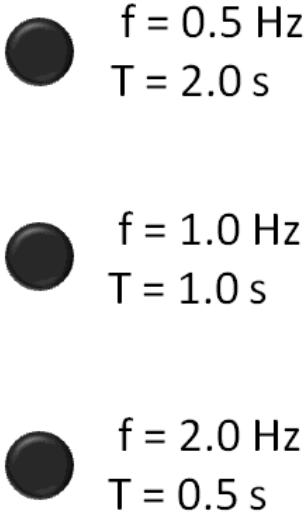
Icon Sources:
analog by I Putu Dicky Adi : https://thenounproject.com/browse/icons/term/analog/
Digital signal by Arthur Shlain: https://thenounproject.com/browse/icons/term/digital-signal

Digitization of Sensor Measurands



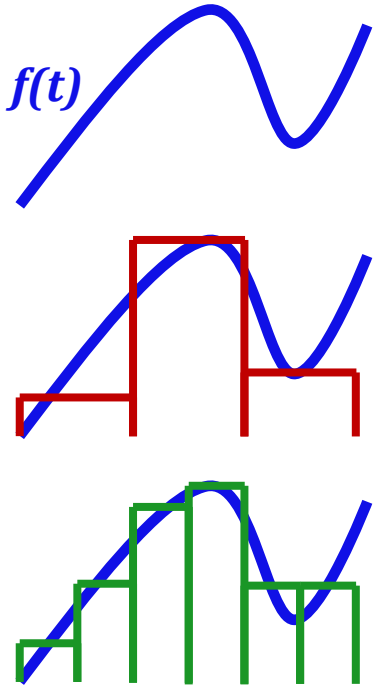
Frequency of signals and measurements

Frequency is the number of occurrences of a repeating event per unit **time**.



Wikimedia Commons

The **sampling frequency** or **sampling rate, f_s,** is the average number of samples obtained in one second (*samples per second*), thus **$f_s = 1/T$**.



The general range of hearing for young people is 20 Hz to 20000 Hz.

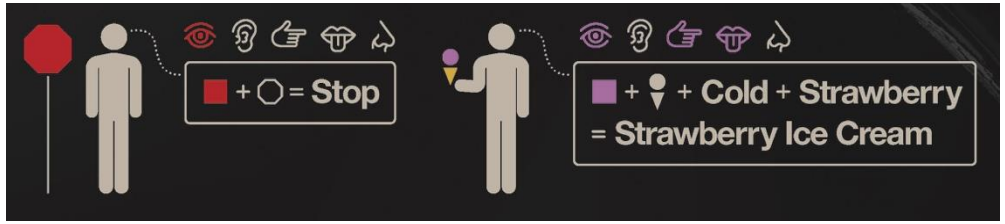
Audio CD, most commonly used with MPEG-1 audio is sampled at 44100 Hz

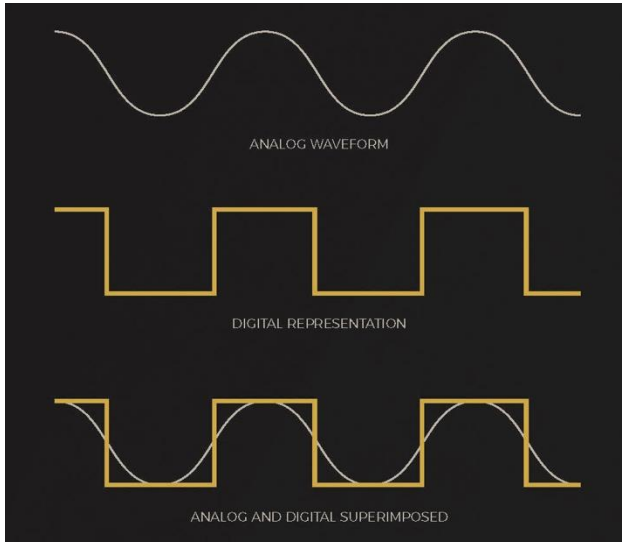
HD DVD (High-Definition DVD) audio tracks are sampled at 98000 Hz

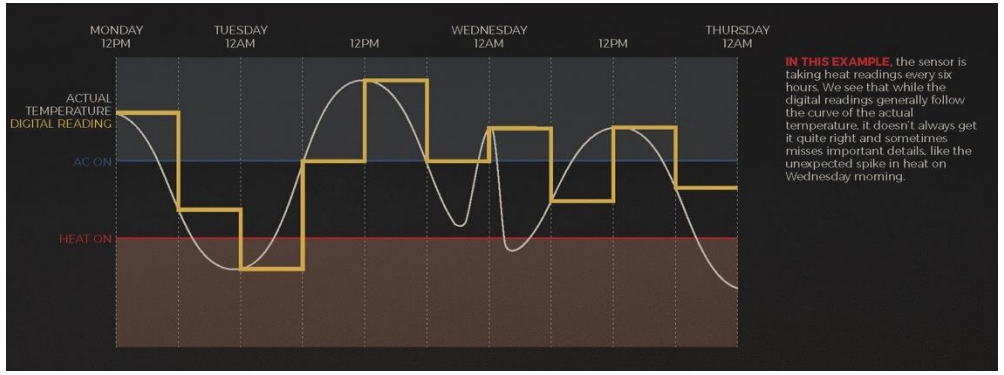
The approximately double-rate requirement is a consequence of the Nyquist theorem.

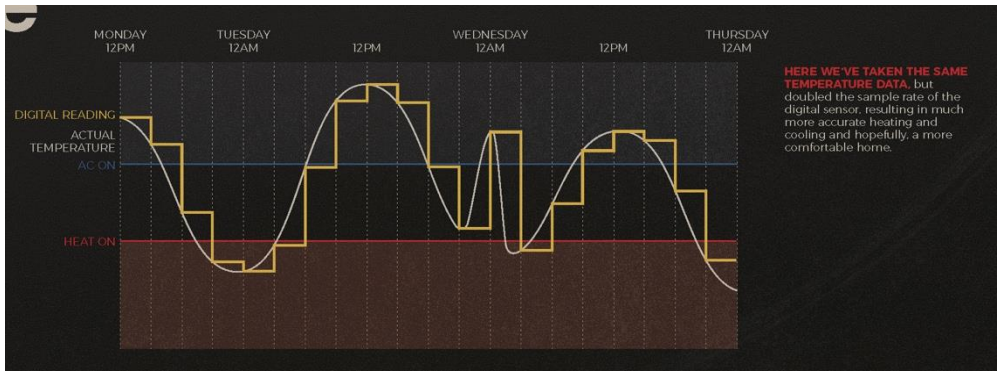
Source:
SparkFun: https://learn.sparkfun.com/

From Analog to the Digital World







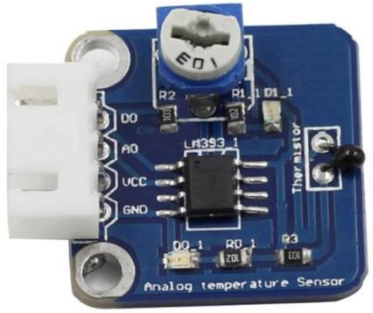


Sources:
McGrath, M. J. and Scanaill, C. N., Sensor Technologies - Healthcare, Wellness and Environmental Applications, Apress Open
Misra, S., Mukherjee, A and Roy, A, Introduction to IoT, Cambridge University Press (2021)
analog by I Putu Dicky Adi : https://thenounproject.com/browse/icons/term/analog/
Digital signal by Arthur Shlain: https://thenounproject.com/browse/icons/term/digital-signal
Sunfounder: http://wiki.sunfounder.cc/index.php?title=Analog_Temperature_Sensor_Module
Sunfounder: https://www.sunfounder.com/products/analog-200nm-370nm-uv-detection-sensor-module

Sensor classification-based on output

Analog sensors

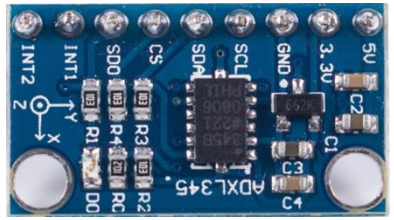
- Output signals are proportional (linearly or non-linearly) to the quantity being measured
- continuous in time and amplitude



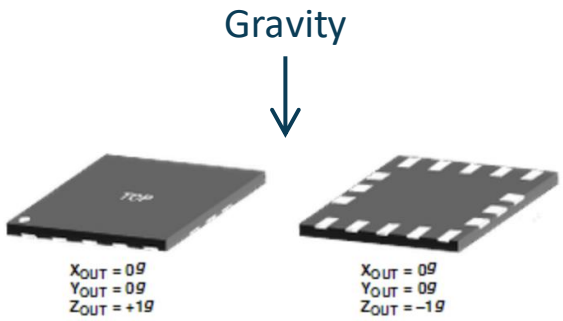
UV detection sensor module

Digital sensors

- Discrete time representation of the quantity being measured
- Binary signals in the form of logic-1 and logic-0
- Output as a single bit (serial transmission) or eight-bit (byte, parallel transmission)



Digital Accelerometer module



Sources:
McGrath, M. J. and Scanaill, C. N., Sensor Technologies - Healthcare, Wellness and Environmental Applications, Apress Opem
Workflow by Nithinan: https://thenounproject.com/browse/icons/term/workflow/

What is a sensor?

A device that receives a stimulus and responds with an electrical signal.

Reference: Fraden, Jacob, Handbook of Modern Sensors, 4th ed. New York: Springer, 2010.

A device that responds to a physical input of interest with a recordable, functionally related output that is usually electrical or optical.

Reference: Jones, Deric P., Biomedical Sensors, 1st ed. New York: Momentum Press, 2010.

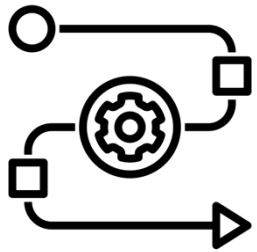
A sensor generally refers to a device that converts a physical measure into a signal that is read by an observer or by an instrument.

Reference: Chen, K. Y., K. F. Janz, W. Zhu, and R. J. Brychta, "Redefining the roles of sensors in objective physical activity monitoring," Medicine and Science in Sports and Exercise, vol. 44, pp. 13–12, 2012.

A device which provides a usable output in response to a specific measurand.

Reference: ANSI, Americian National Standards Institute, "ISA S37.1–1975 (R1982)," ed, 1975.

Key idea



Energy conversion

What is a transducer?

A converter of any one type of energy into another [as opposed to a sensor, which] converts any type of energy into electrical energy.

Reference: Fraden, Jacob, Handbook of Modern Sensors, 4th ed. New York: Springer, 2010.

A sensor differs from a transducer in that a sensor converts the received signal into electrical form only.

A sensor collects information from the real world. A transducer only converts energy from one form to another.

Reference: Khanna, Vinod Kumar, Nanosensors: Physical, Chemical, and Biological. Boca Raton: CRC Press, 2012.

Sources:

McGrath, M. J. and Scanaill, C. N., Sensor Technologies - Healthcare, Wellness and Environmental Applications, Apress Opem

https://www.rohm.com/sensor-shield-support/heart-rate-sensor

Adafruit: https://www.adafruit.com/product/1075?gclid=CjwKCAiA85efBhBbEiwAD7oLQBeaVzL3TCUexZkYZQqnzdLQuRU2b_M6Ygk-0GHJRMqjL_GfdS17tRoC9MwQAvD_BwE

Homedics: https://www.homedics.com/infrared-ear-and-forehead-thermometer/

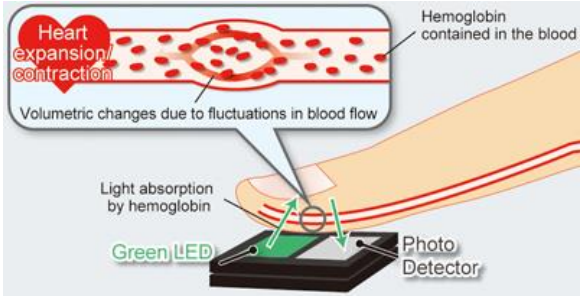
Healthline: https://www.healthline.com/diabetesmine/glucose-test-strips-diabetes-uses-accuracy-costs

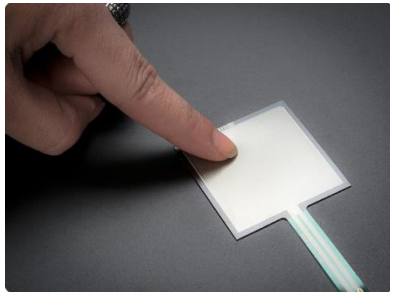
Sensor measurement classification-based on proximity to the measurand

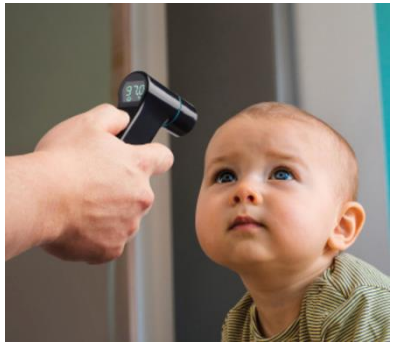
Contact — require physical contact with the quantity of interest.

Non-contact — do not require physical contact with the quantity of interest.

Sample removal — involves an invasive collection of a representative sample by a human or automated sampling system.







Sources:
McGrath, M. J. and Scanaill, C. N., Sensor Technologies - Healthcare, Wellness and Environmental Applications, Apress Open
Misra, S., Mukherjee, A and Roy, A, Introduction to IoT, Cambridge University Press (2021)
Electronics hub: https://www.electronicshub.org/different-types-sensors/

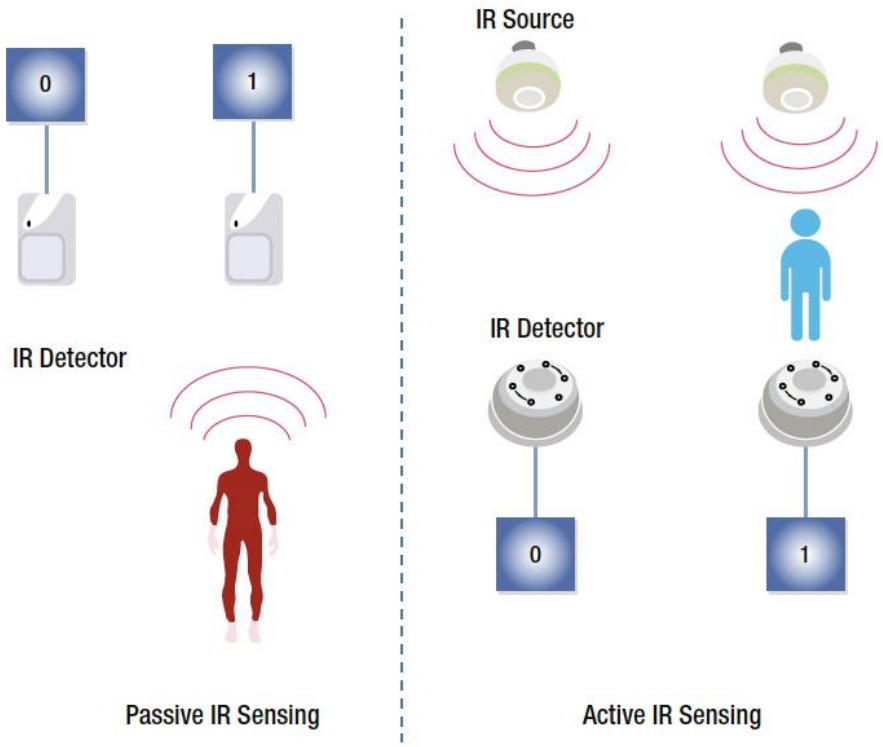
Sensor classification-based on power requirements

Active sensors

- require an external circuitry or mechanism to power them up.

Passive sensors

- do not require external circuitry provide it with power
- directly respond to the external stimuli from its ambient environment and converts it into an output signal.



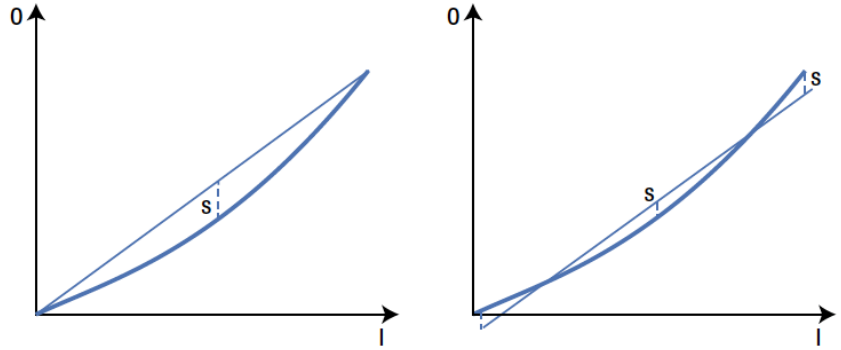
Sensing considerations

Sources:
resolution by Smashicons: https://thenounproject.com/browse/icons/term/resolution/
range by Adrien Coquet: https://thenounproject.com/browse/icons/term/range/
Energy by scarlett mckay: https://thenounproject.com/browse/icons/term/energy/
size by Woof Print Shop: https://thenounproject.com/browse/icons/term/size/
McGrath, M. J. and Scanaill, C. N., Sensor Technologies - Healthcare, Wellness and Environmental Applications, Apress Opem

Resolution:
Minimum discernable measurement

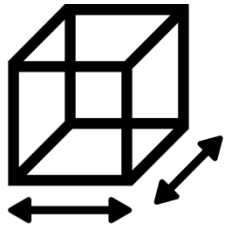
Sensing range:
Maximum and minimum of the possible measurement

Linearity and the transfer function

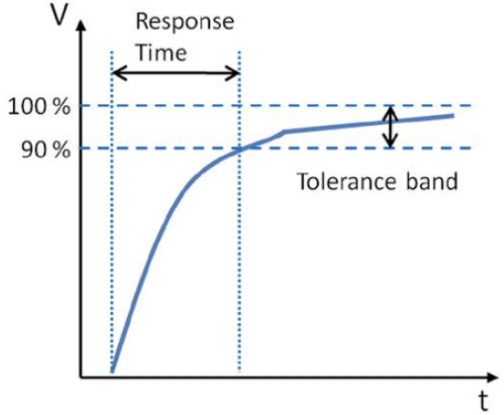


Energy consumption

Device size and form factor



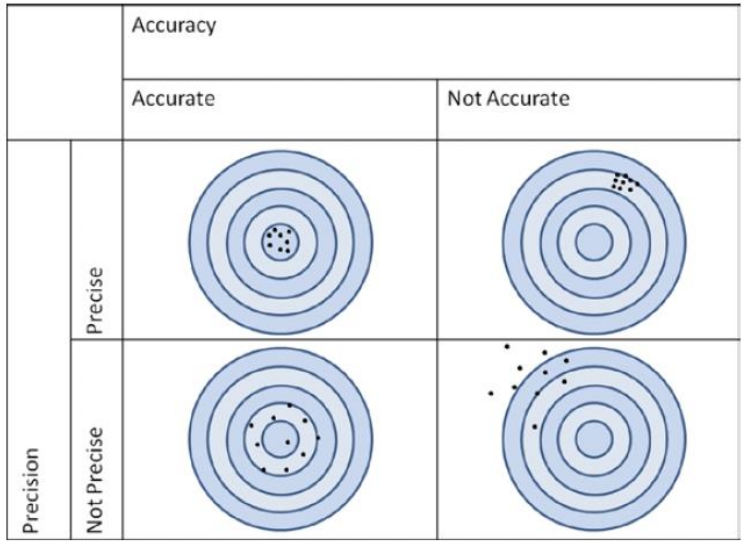
Response time



Sensing errors

Sources:
McGrath, M. J. and Scanaill, C. N., Sensor Technologies - Healthcare, Wellness and Environmental Applications, Apress Open
Misra, S., Mukherjee, A and Roy, A, Introduction to IoT, Cambridge University Press (2021)

Accuracy and Precision



$$Percentage\ Relative\ Error = \frac{(Measured\ Value - True\ Value)}{(True\ Value)} \times 100$$

$$Percentage\ Standard\ Deviation = \frac{(Standard\ Deviation)}{(Mean)} \times 100$$

Systematic Errors

Systematic errors are reproducible inaccuracies that can be corrected with compensation methods, such as feedback, filtering, and calibration

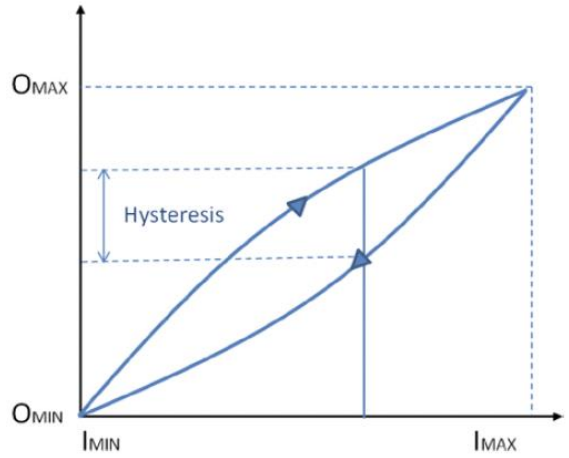
Reference: *Wilson, Jon S., Sensor Technology Handbook. Burlington, MA: Newnes, 2004.*

Random error

Random error (also called noise) is a signal component that carries no information.

The quality of a signal is expressed quantitatively as the signal-to-noise ratio (SNR), which is the ratio of the true signal amplitude to the standard deviation of the noise.

Hysteresis



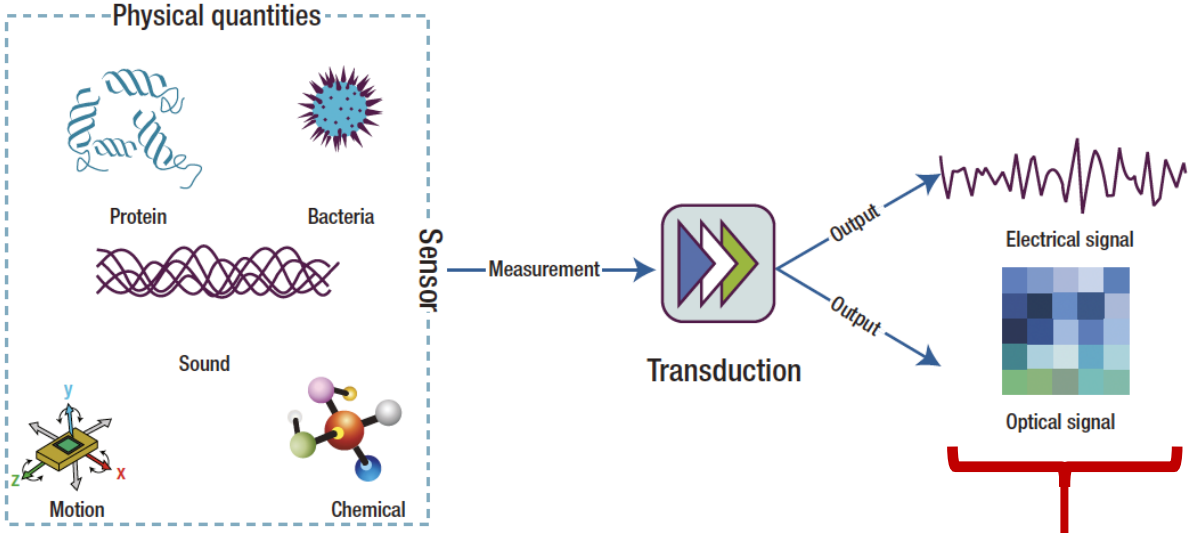
Repeatability

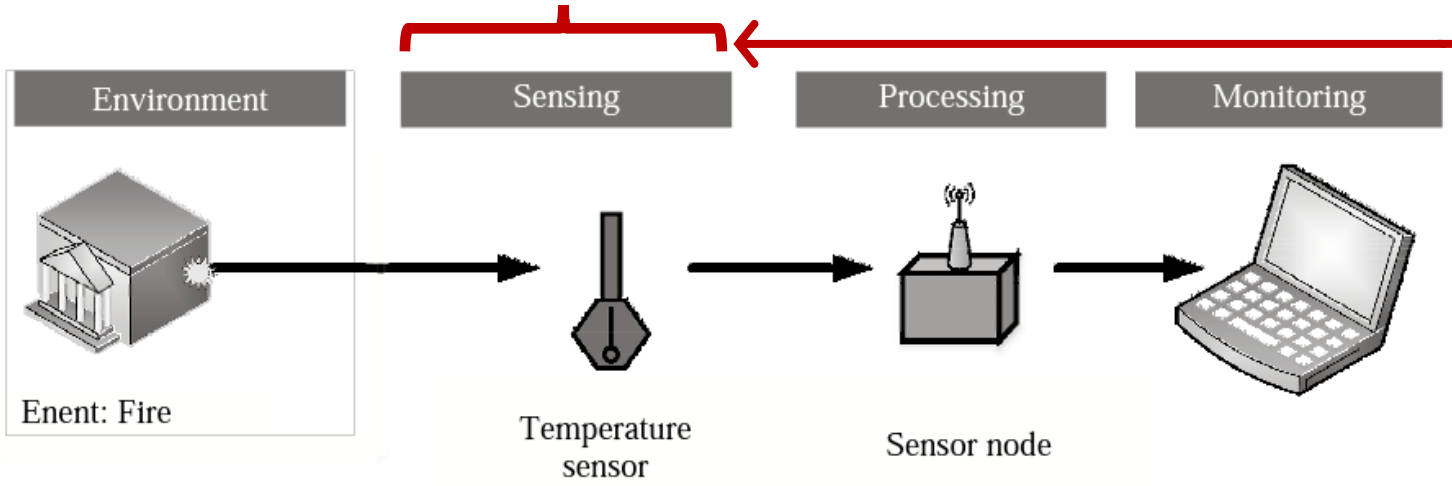
Repeatability is the ability of a sensor to produce the same output when the same input is applied to it.

Sources:
McGrath, M. J. and Scanaill, C. N., Sensor Technologies - Healthcare, Wellness and Environmental Applications, Apress Open
Misra, S., Mukherjee, A and Roy, A, Introduction to IoT, Cambridge University Press (2021)

Outline of a Sensing Operation

Sensor measurements are converted by a transducer into a signal that represents the quantity of interest to an observer or to the external world.

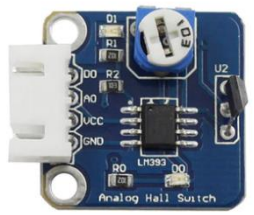




In the IoT-framework, sensor signals and data are communicated to a remote monitor by a processor through a network

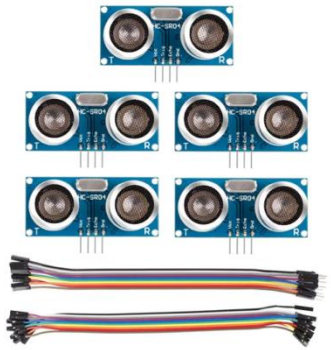
Common types of commercially available sensors (Sunfounder kit)

Source:
Sunfounder:
https://www.sunfounder.com/collections/raspberry-pi-store



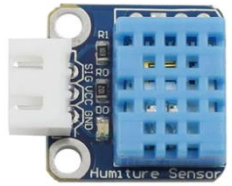
SUNFOUNDER

Analog Hall Sensor Module



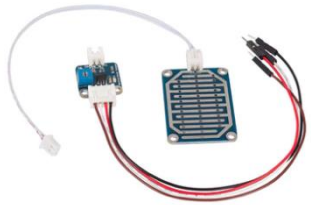
SUNFOUNDER

5pcs HC-SR04 Ultrasonic Module Distance Sensor



SUNFOUNDER

Humiture Sensor Module

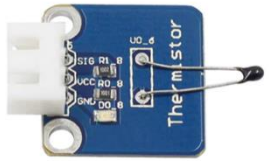


SUNFOUNDER

Raindrop Sensor Module

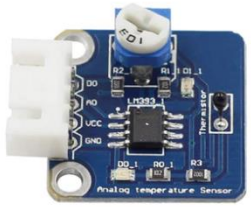
SUNFOUNDER

Photoresistor Sensor Module



SUNFOUNDER

Thermistor Sensor Module

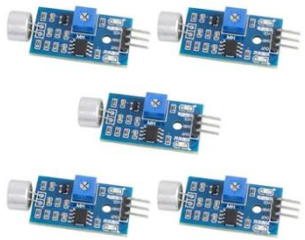


SUNFOUNDER

Analog Temperature Sensor Module

SUNFOUNDER

HHC MQ-2 Gas Sensor Module

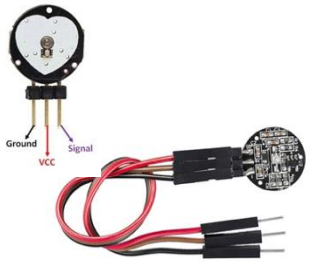


SUNFOUNDER

Sound Sensor Module (5

SUNFOUNDER

Flame Sensor Module

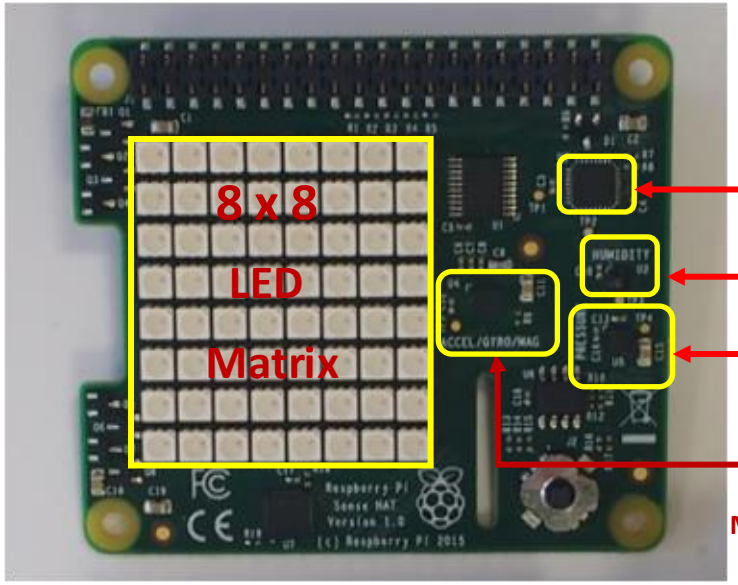


SUNFOUNDER

PulseSensor Heart Rate Monitoring Sensor Module

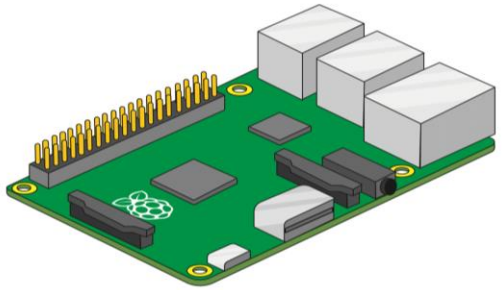
Other types of commercially available sensors and modules

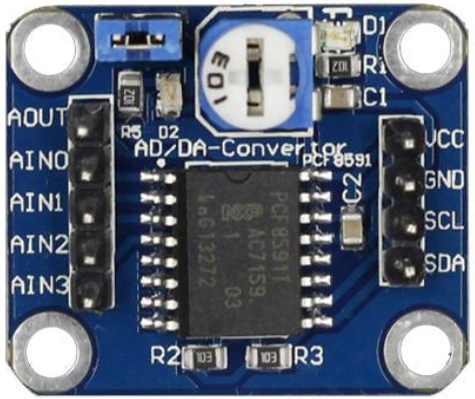
Image and animation source:
https://projects.raspberrypi.org/en/projects/getting-started-with-the-sense-hat/2
http://wiki.sunfounder.cc/index.php?title=Main_Page
https://www.adafruit.com/product/3100#description



- The Sense HAT has a variety of sensors that can be read from:

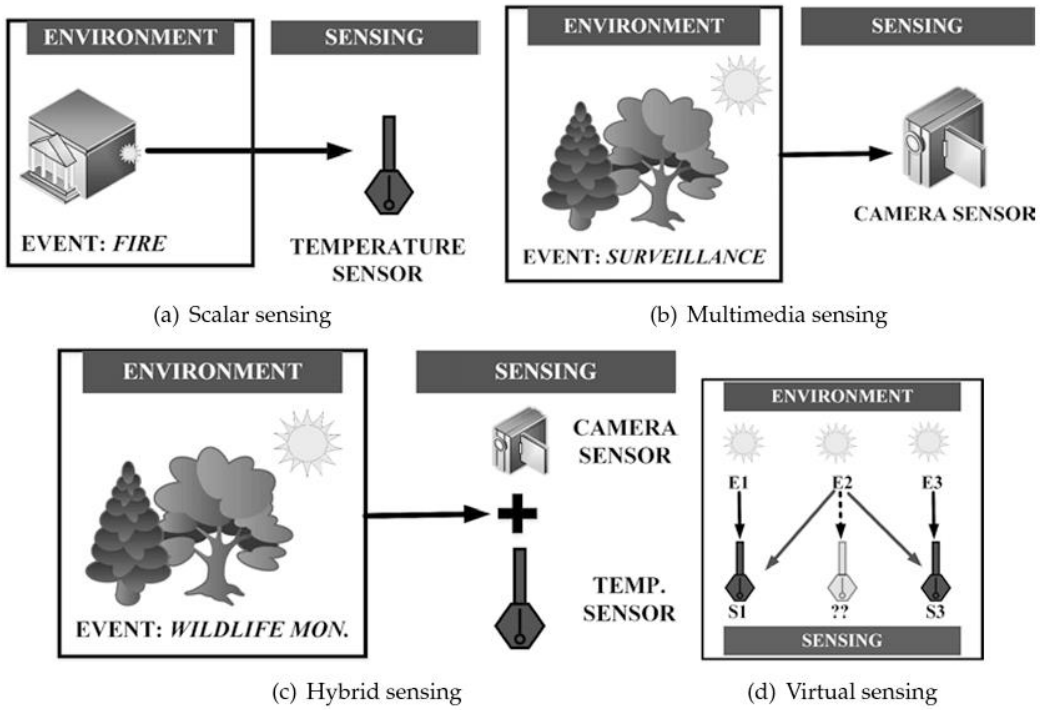
"Temperature"	reads temperature in degrees Celsius
"Humidity"	reads humidity in % RH
"Pressure"	reads atmospheric pressure in millibars
"Rotation"	reads gyroscopic motion in revolutions per second
"Acceleration"	reads acceleration in terms of standard accelerations due to gravity on Earth's surface
"Orientation"	reads orientation relative to magnetic north in degrees
"Magnetic Field"	reads strength and direction of a magnetic field around the sensor in microteslas

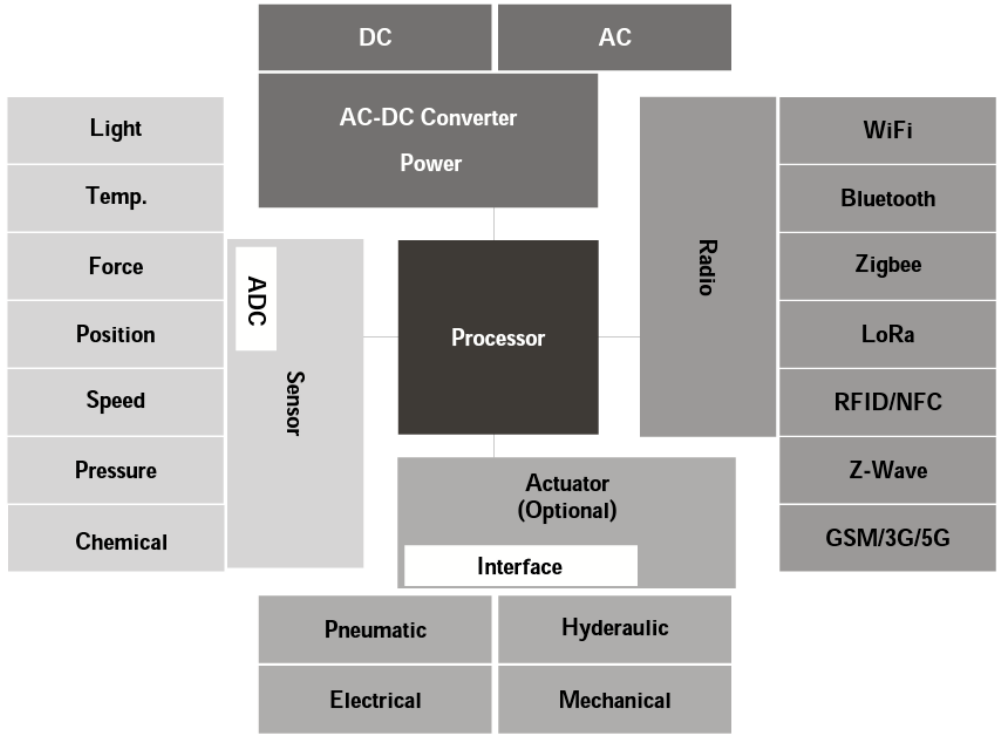




Sources:
Misra, S., Mukherjee, A and Roy, A, Introduction to IoT, Cambridge University Press (2021)

Sensing strategies leading into IoT

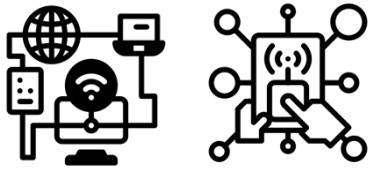




Building the next IoT Architecture

Business-layer

Information-layer



Application

Processing- or Middleware-layer

Data processing

Communication-layer

Data transfer

Sensor-layer

Things

Icon Sources:

sensor by Carolina Cani:, sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo: https://thenounproject.com/browse/icons/term/sensor

fire sensor by LAFS : https://thenounproject.com/browse/icons/term/fire-sensor/

Ultrasound by Shocho: https://thenounproject.com/browse/icons/term/ultrasound/

Network by Solikin:, Network by Tippawan: https://thenounproject.com/browse/icons/term/network

application by Chaowalit Koetchuea: https://thenounproject.com/browse/icons/term/application

wifi network by ProSymbols: https://thenounproject.com/browse/icons/term/wifi-network/

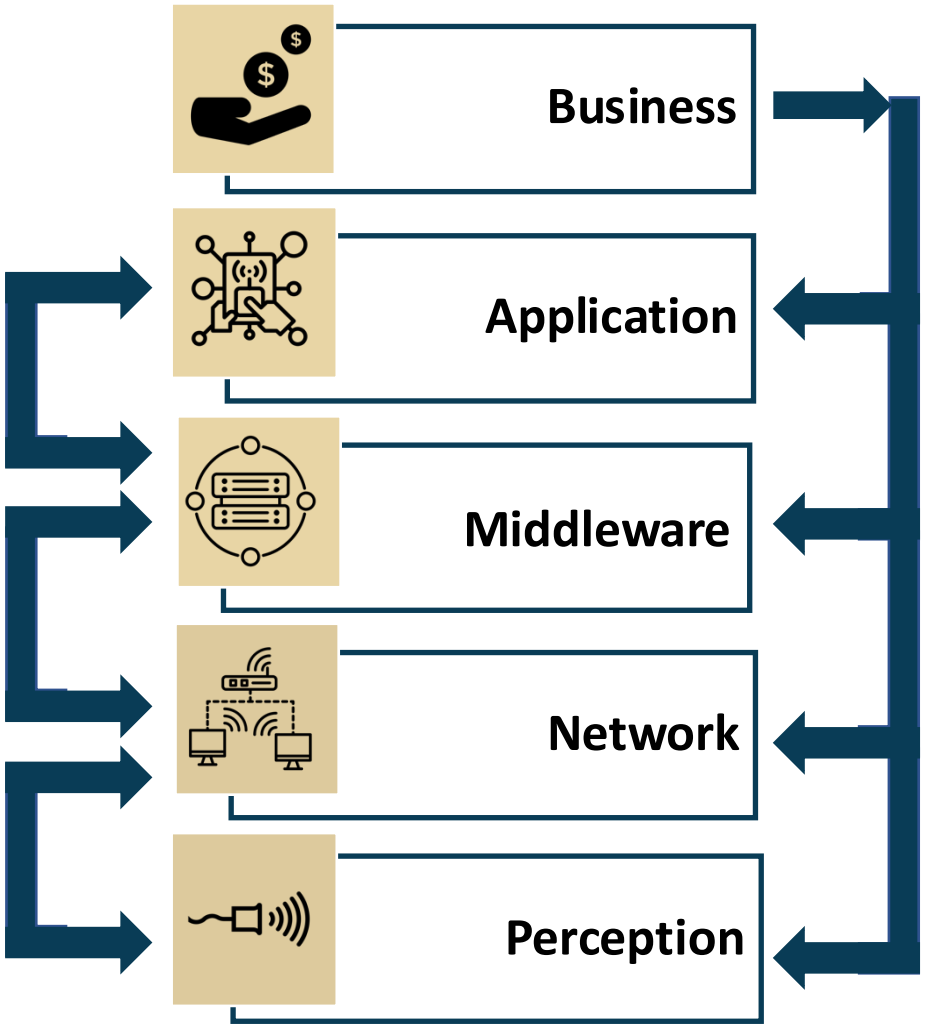
data transfer by Jajang Nurrahman: https://thenounproject.com/browse/icons/term/data-transfer/

transfer data by tezar tantular: https://thenounproject.com/browse/icons/term/transfer-data/

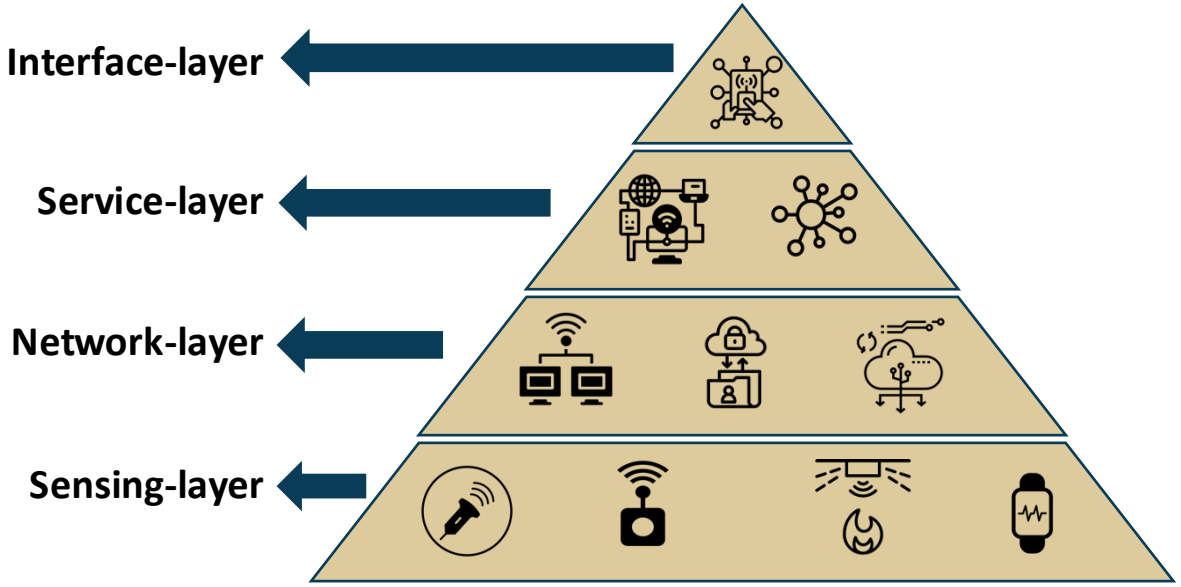
data processing by Jajang Nurrahman: https://thenounproject.com/browse/icons/term/data-processing

Business by DinosoftLab: https://thenounproject.com/browse/icons/term/business/

The 5-Layer IoT Architecture



Service-oriented IoT Architecture

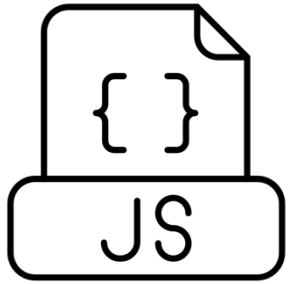


Sources:
sensor by Carolina Cani:, sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo: https://thenounproject.com/browse/icons/term/sensor
wifi network by Matthias Hartmann:: https://thenounproject.com/browse/icons/term/wifi-network/
application by Chaowalit Koetchuea: https://thenounproject.com/browse/icons/term/application/
IoT Architecture layers: https://www.startertutorials.com/blog/iot-architecture-layers.html

Source:
https://flask.palletsprojects.com/en/2.2.x/
https://en.wikipedia.org/wiki/Plotly
javascript file by SAM Designs from https://thenounproject.com/browse/icons/term/javascript-file/
html by tezar tantular from https://thenounproject.com/browse/icons/term/html/

Demo project:
Create a Flask API for IoT Applications
[Graded Lab Activity]

1. Python 3 with any IDE or terminal
2. Familiarity with flask API
3. Basic HTML with JavaScript
4. Familiarity with Plotly



School of Engineering & Applied Science
THE GEORGE WASHINGTON UNIVERSITY

GW

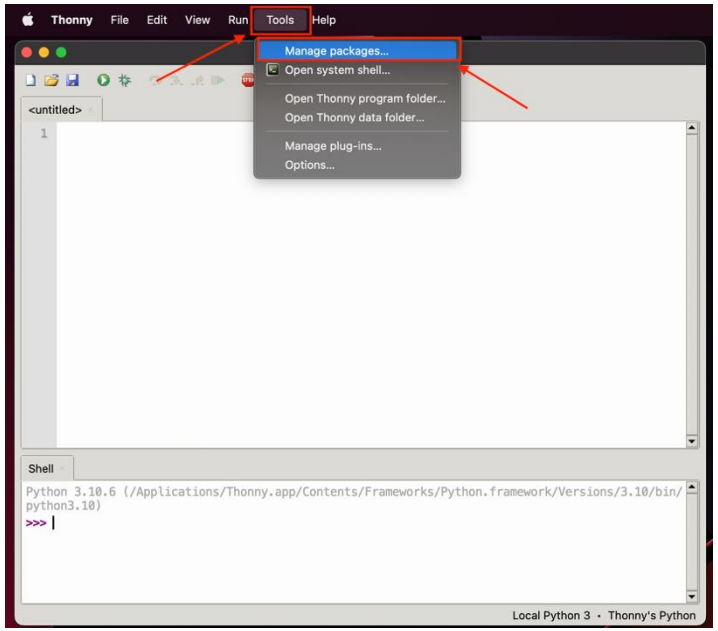
Prof. Kartik Bulusu, MAE Dept.
Spring 2026
MAE 6291: Internet of Things for Engineers and Scientists

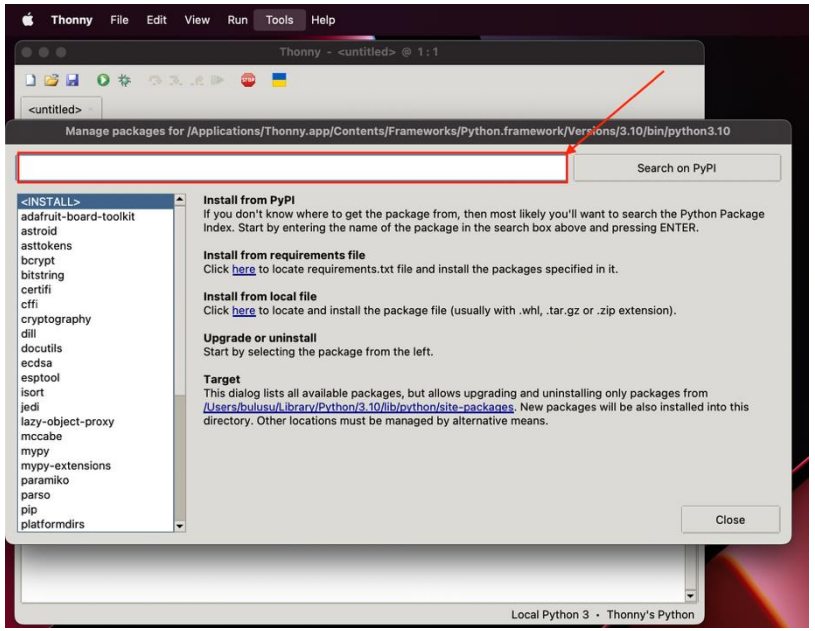
Source:
https://flask.palletsprojects.com/en/2.2.x/
https://en.wikipedia.org/wiki/Plotly

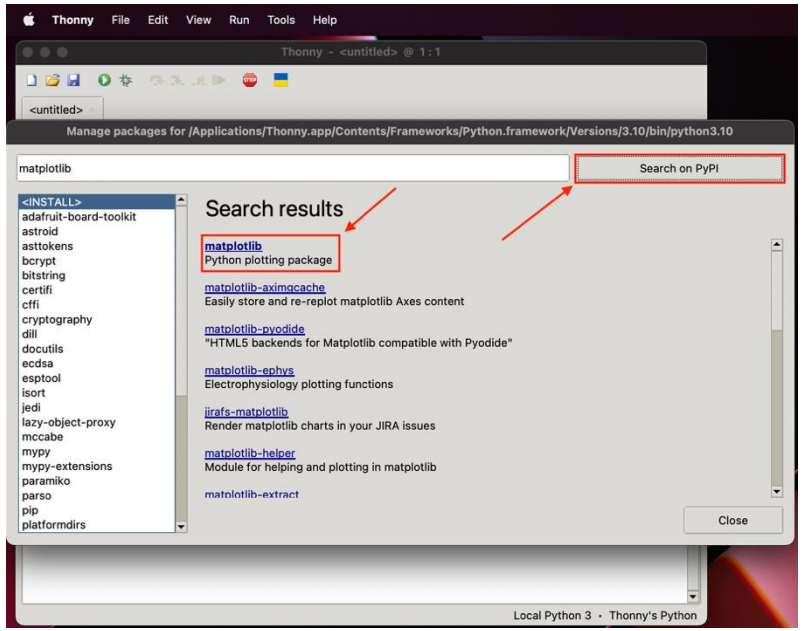
- **Flask** is a micro web framework written in Python.
- **Components:**
  - ***Werkzeug:*** Utility library for Web Server Gateway Interface (WSGI) applications
  - ***Jinja:*** Template engine similar to Django that handles templates in a sandbox
  - ***MarkupSafe:*** String handling library
  - ***ItsDangerous:*** Safe data serialization library

- **Plotly** provides online graphing, analytics, and statistics tools
  - for individuals and collaboration,
  - as well as scientific graphing libraries for Python, R, MATLAB, Perl, Julia, Arduino, and REST.
- **Open-source products:**
  - ***Dash:*** Open-source framework for building web-based analytic applications.
  - ***Chart Studio Cloud:*** Free, online tool for interactive graphics
  - ***Plotly.js data visualization JavaScript library*** for creating graphs and powers Plotly.py for Python, as well as Plotly.R for R, MATLAB, Node.js, Julia, and Arduino and a REST API

Installing packages in Thonny







Check or install the following libraries in Python 3.10.11: (Note these are the bare minimum versions)

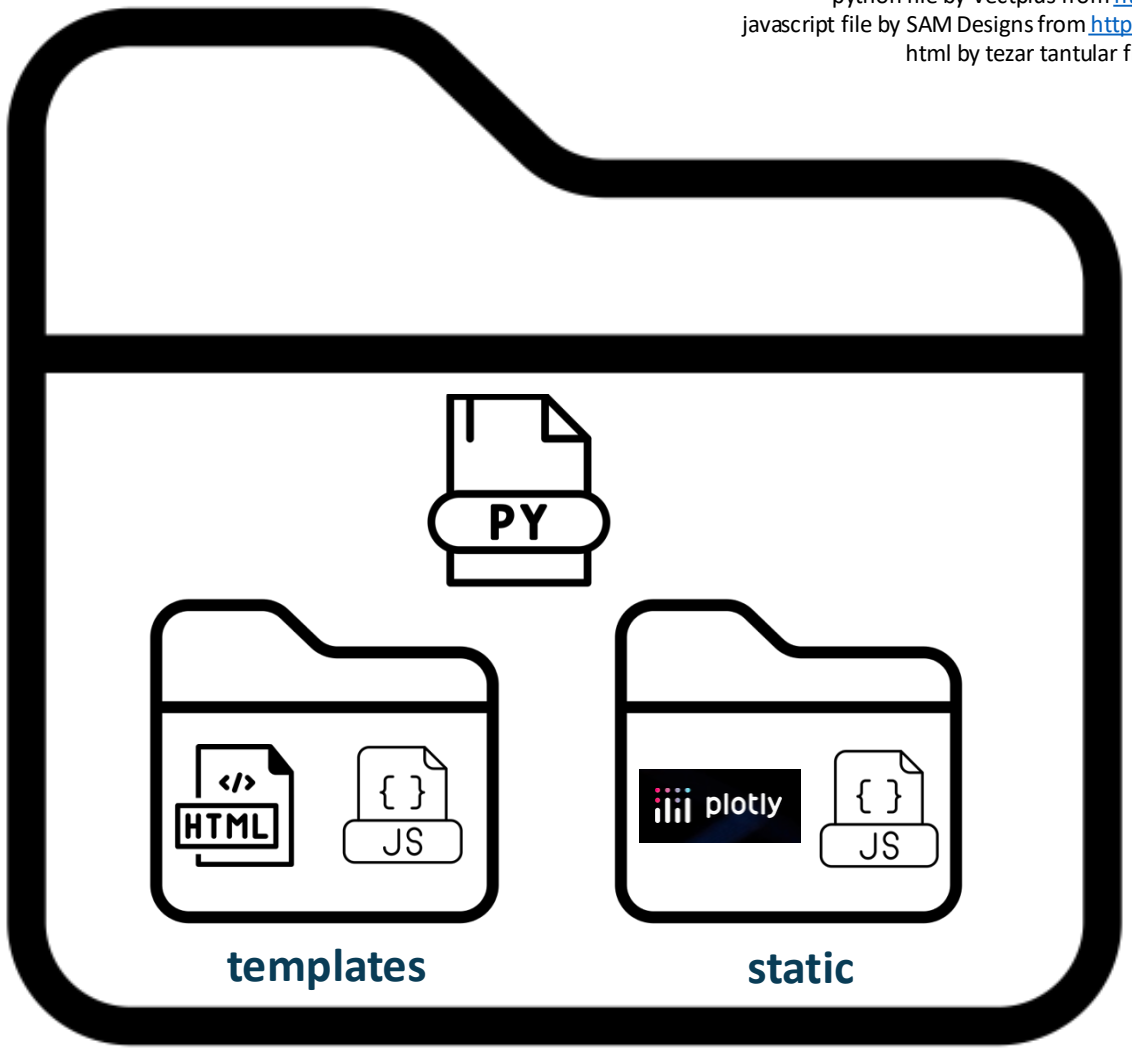
Flask 2.3.1
plotly 5.15.0
simplejson 3.19.1
pandas 2.0.2
numpy 1.24.2
matplotlib 3.7.1
datetime 5.0
time 1.0.0

Alternative:

```
>>> pip install <package_name>
>>> # Or install using pip3
>>> # in your virtual environment
```

Essentials: Files and folders needed to create the intended APP

Source:
Folder by Colourcreatype from https://thenounproject.com/browse/icons/term/folder/
python file by Vectplus from https://thenounproject.com/browse/icons/term/python-file/
javascript file by SAM Designs from https://thenounproject.com/browse/icons/term/javascript-file/
html by tezar tantular from https://thenounproject.com/browse/icons/term/html/
https://en.wikipedia.org/wiki/Plotly



Skeleton of the Python program for a flask-server on the Raspberry Pi

Flask
web development, one drop at a time

Source:
https://flask.palletsprojects.com/en/2.2.x/

Step-1:
Initialize variables and setup Flask instance

```
from flask import Flask, request, render_template
from flask_restful import Resource, Api, reqparse, inputs
import pandas as pd
import json
import plotly
import plotly.subplots
import plotly.express as px
import random
import numpy as np
import matplotlib.pyplot as plt
import time
import datetime
import logging
import thing_file
```

Import libraries that are relevant for interaction with the Raspberry Pi hardware such as

- flask,
- json,
- plotly and its derivatives
- pandas
- numpy
- matplotlib etc.

```
# ======== Initialize Logging ===========

# Global logging configuration
logging.basicConfig(level=logging.WARNING)

# Logger for this module
logger = logging.getLogger('main')

# Debugging for this file.
logger.setLevel(logging.INFO)
```

Source:
Folder by Colourcreatype from https://thenounproject.com/browse/icons/term/folder/
python file by Vectplus from https://thenounproject.com/browse/icons/term/python-file/
javascript file by SAM Designs from https://thenounproject.com/browse/icons/term/javascript-file/
html by tezar tantular from https://thenounproject.com/browse/icons/term/html/
https://en.wikipedia.org/wiki/Plotly

Provides warning on any components that work within flask or other imported libraries

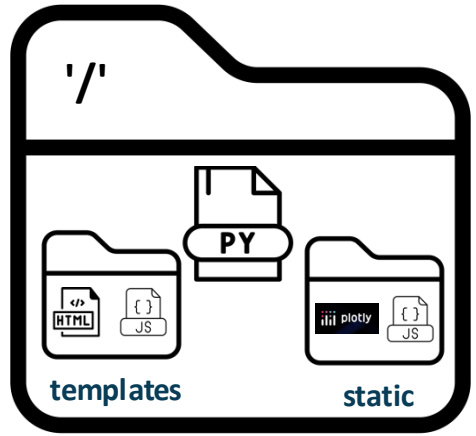
Reference:
https://docs.python.org/3/howto/logging.html

thing_file.py
Logging libraries and a few more are place in this custom library provided to you

Step-2:
Initialize variables and setup Flask instance

```
# ==== Flask & Flask-RESTful instance variables ====

# Core Flask app.
app = Flask(__name__)

# ==== Flask & Flask-Restful Related Functions ====
# @app.route applies to the core Flask instance (app).
# Here we are serving a simple web page.
@app.route('/' + thing_file.thing_name)
```

Initiate flask server-related variables and functions



```
def notdash():
    global data
    data = {
        'timeT': [],
        'Voltage': []
    }
```

Dictionary of empty lists that get appended with data

Step-3:
Create a function notdash() to return JSON-formatted data to an html frontend

```
    # Create the graph with subplots
        fig = plotly.tools.make_subplots(rows=1, cols=1, vertical_spacing=0.2)
        fig['layout']['margin'] = {
            'l': 30, 'r': 10, 'b': 30, 't': 10
        }
```

Dictionary of figure layout that is transferred to the html frontend with plotly, JavaScript embedded in it

```
        for i in range(20):
                    data['Voltage'].append(random.randint(0, 100))
                    data['timeT'].append(timeT)

            fig.append_trace({
              'x': data['timeT'],
              'y': data['Voltage'],
                        'mode': 'lines+markers',
              'type': 'scatter'
            }, 1, 1)
```

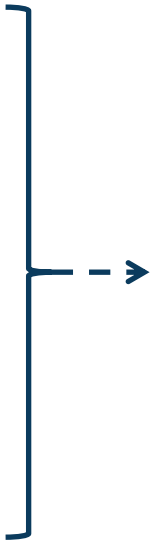
Loop to generate random data and plot it in a trace that is transferred to the html frontend with plotly, JavaScript embedded in it

```
graphJSON = json.dumps(fig, cls=plotly.utils.PlotlyJSONEncoder)
return render_template('notdash.html', graphJSON=graphJSON)
```

json.dumps will convert a subset of Python objects into a json
render_template tells Flask to use an HTML template

Step-5
Create a function notdash() to return JSON-formatted data to an html frontend

```
if __name__ == '__main__':

    # If you have debug=True and receive
    # the error "OSError: [Errno 8] Exec format error", then:
    # remove the execuition bit on this file from a Terminal, ie:
    # chmod -x flask_api_server.py
    #
    # Flask GitHub Issue:
    # https://github.com/pallets/flask/issues/3189

    app.run(host="0.0.0.0", debug=True)
```



Creates entry point into the program and executes app.run() in debug mode.

debug=False:
translates to developer mode

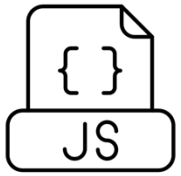
app.run() renders the webpage with data plots on a

local host:
127.0.0.1:5000

Port:5000 is a default

The host address can be changed to the IP address of the server.

Skeleton of the the basic HTML code to display data from your flask-app



Sources:
https://plotly.com
API by Vectors Point from https://thenounproject.com/browse/icons/term/api/
json by ME from https://thenounproject.com/browse/icons/term/json/
javascript file by SAM Designs from https://thenounproject.com/browse/icons/term/javascript-file/
html by tezar tantular from https://thenounproject.com/browse/icons/term/html/
https://towardsdatascience.com/web-visualization-with-plotly-and-flask-3660abf9c946

```
<!doctype html>
<html>

<head>
        <meta http-equiv="refresh" content="10">
</head>

<body>
        <h1>Prof. Kartik Bulusu's sensor data</h1>
        <div id='chart' class='chart'"></div>
</body>

<!-- <script src='https://cdn.plot.ly/plotly-latest.min.js'></script> -->

<script src='/static/plotly-latest.min.js'></script>

<script type='text/javascript'>
        var graphs = {{graphJSON | safe}};
        Plotly.plot('chart',graphs,{});
</script>

</html>
```

Will refresh the page every 10 seconds

Webpage title etc

Location of plotly-latest.min.js

To download: https://plotly.com/javascript/getting-started/

{{graphJSON | safe}}: Injects a variable that came from the server directly in the JavaScript code.

Plotly.plot(): Creates a **line chart** drawn into a element on the page, with data from **graphs** with **layout** provided by the server

HW on Ultrasound sensor due on 02/26/2025 (10 points)

git clone https://github.com/gwu-mae6291-iot/spring2025_codes.git

Goal-1

Set up a Python virtual environment
- "Sandbox" your Python project

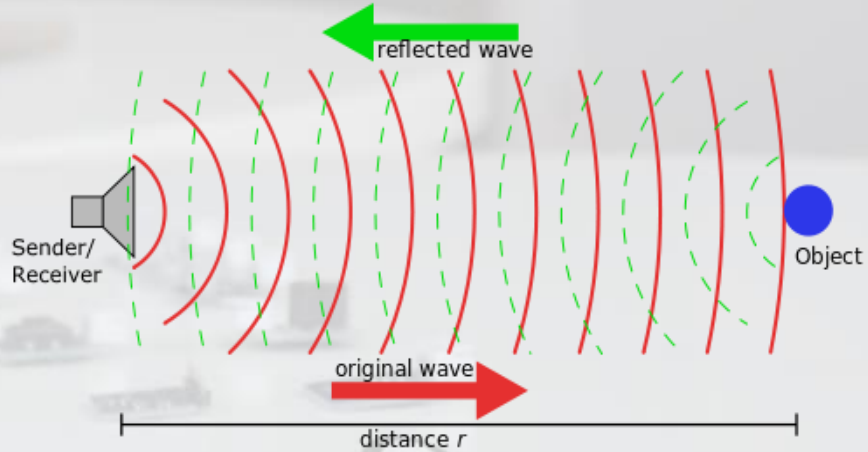
Goal-2

Demonstrate your python script at boot
Using cron, the UNIX scheduler

Ultrasound Signals and its Applications

Icon Sources:
sensor by Carolina Cani:, sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo: https://thenounproject.com/browse/icons/term/sensor
fire sensor by LAFS : https://thenounproject.com/browse/icons/term/fire-sensor/
Ultrasound by Shocho: https://thenounproject.com/browse/icons/term/ultrasound/
Network by Solikin:, Network by Tippawan: https://thenounproject.com/browse/icons/term/network
application by Chaowalit Koetchuea: https://thenounproject.com/browse/icons/term/application/

Source: https://youtu.be/J-VVh5ezqGA?si=G9iu055uV3MinyUA



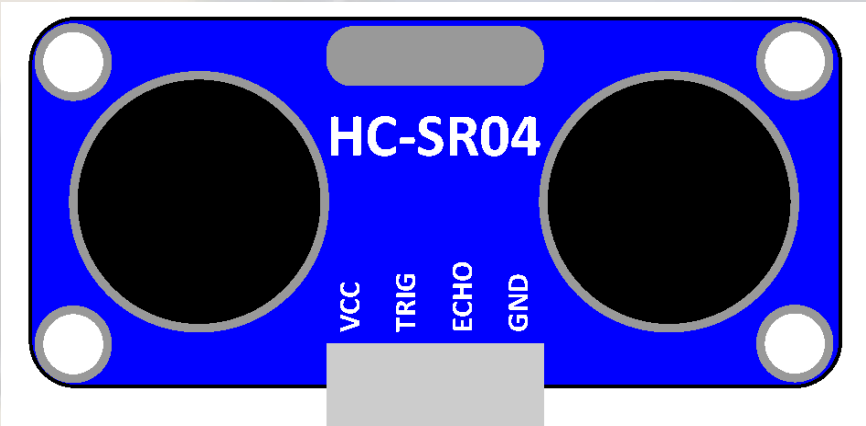
$$Distance\ traversed = (Speed\ of\ sound) \times (Time\ elapsed/2)$$

Photo: Kartik Bulusu

School of Engineering & Applied Science
THE GEORGE WASHINGTON UNIVERSITY

GW

Know your Ultrasonic Sensor



The Ultrasonic sensor sends out ultrasonic waves to detect objects and measure distances.

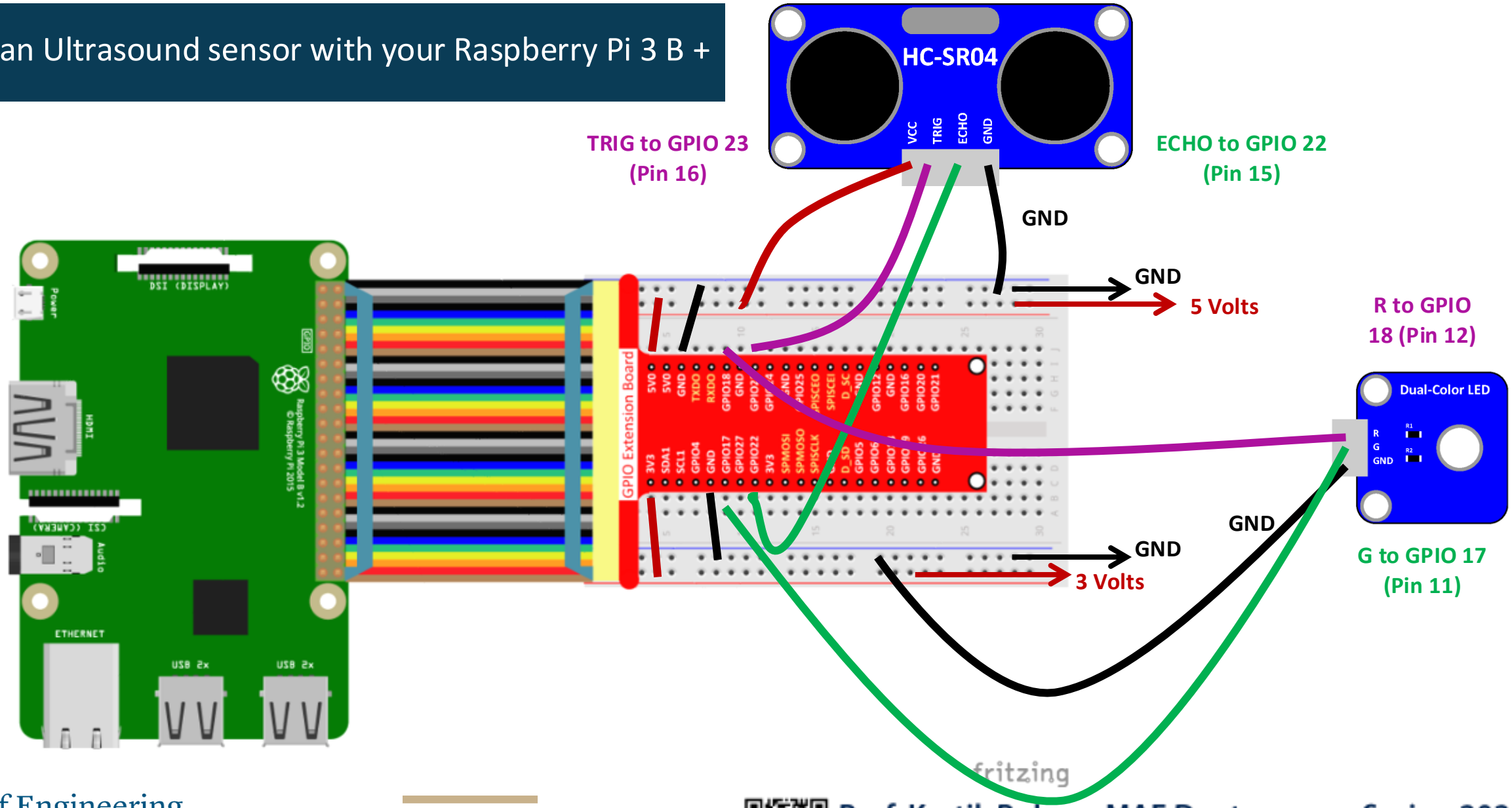
Connectors:
4-pin connector wires
Or in some cases 5-pin connector wires

Goal of this HW segment:

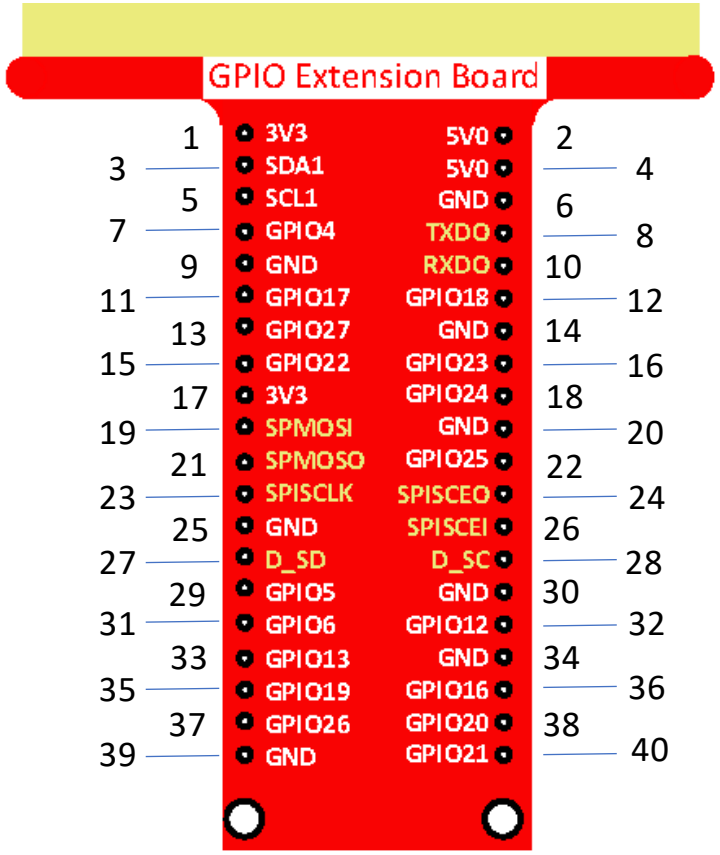
- Co-work (if you need to)
  - Observe, ask and try in groups
- Make
  - Build-a-hack
  - Ultrasound sensors and Raspberry Pi 4B boards
- Analyze data using Python

Photo: Kartik Bulusu

Start an Ultrasound sensor with your Raspberry Pi 3 B +



Pseudo-code to kick start your Raspberry Pi Model 3 B+ (RPi)




```
import LIBRARY as NAME
import ANOTHER_LIBRARY
```

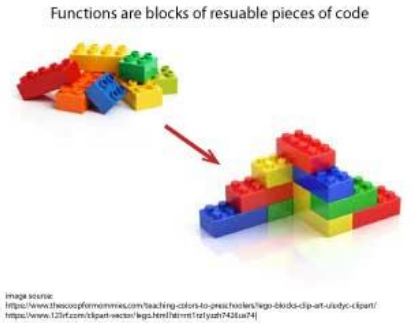
```
INITIALIZE GPIO CHANNELS

DEFINE SETUP FUNCTION
    GPIO.setmode(GPIO.BOARD)
    GPIO.setup(CHANNEL-1, GPIO.OUT)
    GPIO.setup(CHANNEL-2, GPIO.IN))
```

```
DEFINE DISTANCE FUNCTION
return (TIME_ELAPSED / 2) * 340 * 100
```

```
DEFINE LOOP FUNCTION
    while True:
        ...
```

```
DEFINE DESTROY FUNCTION
    CLEAN UP GPIO CHANNELS
```

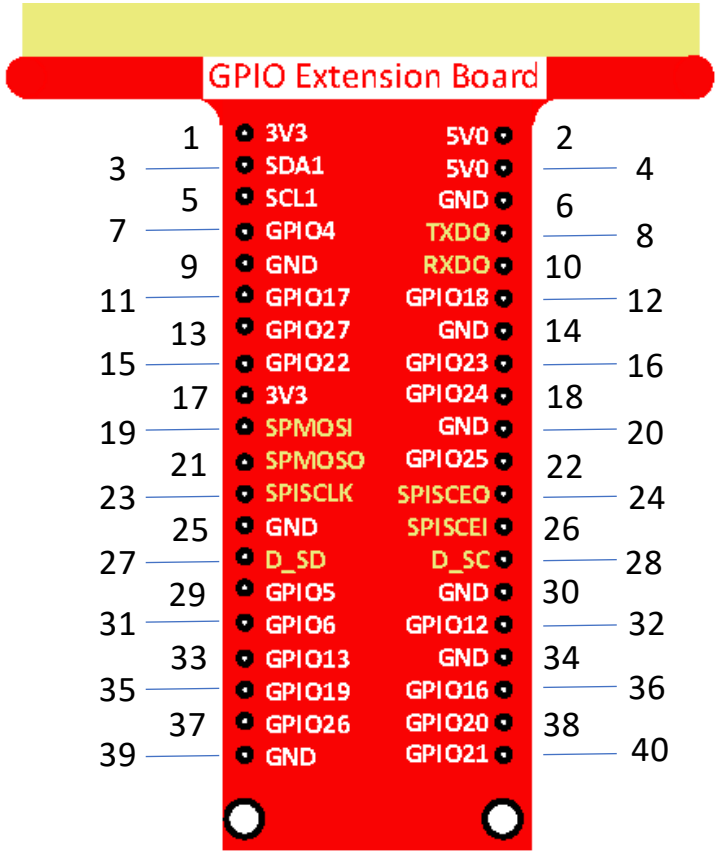
User defined functions




```
if __name__ == "__main__":
    setup():
    try:
        loop()
    except KeyboardInterrupt:
        destroy()
```

Entry point into the program – pulls in all user defined functions

A simple python code to kick start your Raspberry Pi Model 3 B+ (RPi)




```
import RPi.GPIO as GPIO
import time
```

```
TRIG = 16
ECHO = 15
def setup():
    GPIO.setmode(GPIO.BOARD)
    GPIO.setup(TRIG, GPIO.OUT)
    GPIO.setup(ECHO, GPIO.IN))
```

```
def distance():
    GPIO.output(TRIG, 0)
    time.sleep(0.000002)
    GPIO.output(TRIG, 1)
    time.sleep(0.00001)
    GPIO.output(TRIG, 0)

    while GPIO.input(ECHO) == 0:
        time1 = time.time()

    while GPIO.input(ECHO) == 1:
        time2 = time.time()

    during = time2 - time1
    return (during / 2) * 340 * 100
```

```
def loop():
    while True:
        dist = distance()
        print(dist, 'cm')
        print('')
        time.sleep(0.1)
```

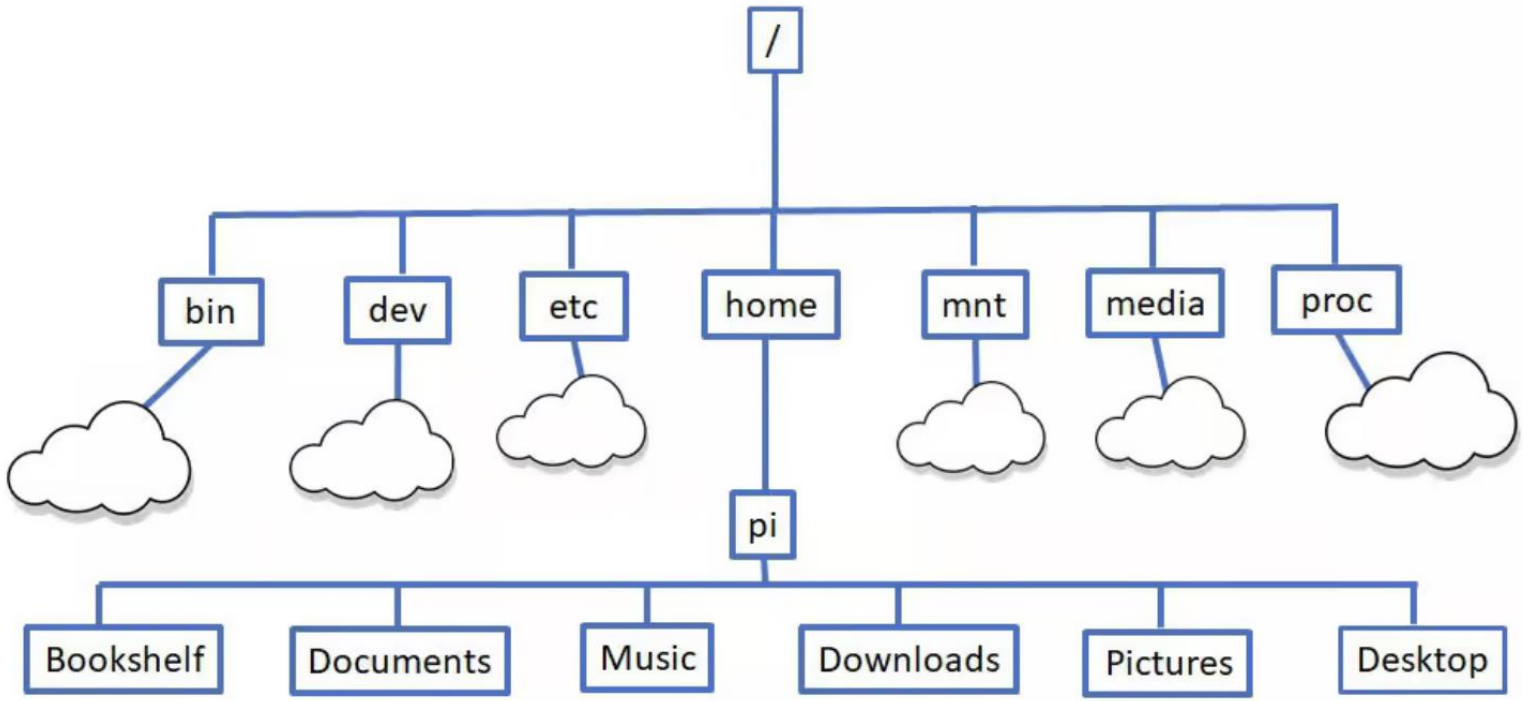
```
def destroy():
    GPIO.cleanup()
```

```
if __name__ == "__main__":
    setup():
    try:
        loop()
    except KeyboardInterrupt:
        destroy()
```


Photo: Kartik Bulusu
Someone should summarize what we learned today

Additional slides: The Linux Terminal

Source:
https://www.circuitbasics.com/how-to-navigate-the-raspberry-pi-file-system/



Explore files and folders

Command	Description
pwd	Show the **current** directory you are in
ls	List files in the current directory; add -l or -a for more detail or to see hidden files
cd /path/goes/here	Change directory (for example cd /home/pi/Documents)
mkdir myfolder	Create a new directory named myfolder
cp file1 file2	Copy a file; cp -r dir1 dir2 copies a whole directory
mv old new	Move or rename files/directories
rm file	Delete a file; use rm -r folder carefully to remove a directory

Helpful navigation and history tips

Command or Keyboard Entry	Description
clear	Clear the terminal screen.
history	Show a numbered list of past commands; run one again with !number.
Up/Down arrow keys	Scroll through previously used commands to run or edit them.

View and edit files

Command	Description
nano file.txt	Open a simple text editor in the terminal to create or edit a file
less file.txt	View a long file one page at a time (use q to quit)
cat file.txt	Print the contents of a text file to the screen

System information and status

Command	Description
hostname -I	Show the Pi's IP address
uname -a	Show Linux kernel and system details
df -h	Show disk space usage in human-readable form
free -h	Show memory (RAM) usage
uptime	See how long the Pi has been running
top or htop	Monitor running processes and CPU usage (press q to exit top).

IP address of your RPi connected to the network using WiFi

ifconfig -a

```
pi@raspberrypi017: ~
File  Edit  Tabs  Help
pi@raspberrypi017:~ $ ifconfig -a
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether ea:e0:b9:19:85:8f  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 169.254.247.201  netmask 255.255.0.0  broadcast 169.254.255.255
        inet6 fe80::21e4:d41c:791a:a9d7  prefixlen 64  scopeid 0x20<link>
        ether dc:a6:32:98:38:24  txqueuelen 1000  (Ethernet)
        RX packets 1874  bytes 166519 (162.6 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1723  bytes 1258105 (1.1 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 53  bytes 5297 (5.1 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 53  bytes 5297 (5.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.198.24.23  netmask 255.255.252.0  broadcast 10.198.27.255
        inet6 fe80::3d8a:ccb:6a3a:2f0  prefixlen 64  scopeid 0x20<link>
        ether dc:a6:32:98:38:25  txqueuelen 1000  (Ethernet)
        RX packets 12  bytes 1379 (1.3 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 67  bytes 8066 (7.8 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

pi@raspberrypi017:~ $
```